

RESEARCH

Open Access



Automating fuzz driver generation for deep learning libraries with large language models

Tianming Zheng¹, Fanchao Meng¹, Ping Yi¹ and Yue Wu^{1*}

Abstract

The widespread adoption of deep learning (DL) libraries has raised concerns about their reliability and security. While prior works leveraged large language models (LLMs) to generate test programs for DL library APIs, the hardcoded program behaviors and low code validity rates render them impractical for real-world testing. To address these challenges, we propose FD-FACTORY, a fully automated framework that leverages LLMs to generate fuzz drivers for DL API testing. The fuzz driver programs accept mutated inputs from fuzzing engines to achieve effective code analysis. Inspired by the modular design of industrial production lines, FD-FACTORY decomposes the generation process into eight distinct stages: *Preparation, Initial Fuzz Driver Generation, Early Stop Checks, Verification, Issue Diagnosis, Decision Making, Repair Loop, and Deployment*. Each stage is handled by dedicated agents or tools to enhance construction efficiency. Experimental results demonstrate that FD-FACTORY achieves 73.67% and 65.33% success rates in generating fuzz drivers for PyTorch and TensorFlow, producing an improvement of 34.66 to – 54.66% than existing approaches. In addition, FD-FACTORY provides more comprehensive coverage tracking by supporting both Python and native C/C++ code. It achieves a total coverage of 308,351 lines on PyTorch and 528,427 lines on TensorFlow, substantially surpassing the results reported by previous approaches. Unlike prior approaches relying on repeated interactions with the LLM servers throughout the entire testing process, our framework confines the use of LLMs strictly to the fuzz driver generation stages before deployment. Once generated, the fuzz drivers can be reused without further LLM involvement, thereby enhancing the practicality and sustainability of LLM-assisted fuzzing in real-world scenarios.

Keywords Fuzz driver generation, Large language model, Deep learning library, Vulnerability detection

Introduction

As the foundation of artificial intelligence (AI) applications, deep learning (DL) libraries provide software implementations for a variety of DL operations and algorithms (Archana and Jeevaraj 2024; Joshi et al. 2025; Kheddar et al. 2024; GoogleCloud 2025; Tableau 2025; Vidhya 2023). However, like conventional codebases, DL libraries inevitably exhibit inherent security issues (Zhang et al. 2025). This includes common program bugs such as incorrect program functionalities,

numerical errors, and producing NaN (Guo et al. 2020; Christou et al. 2023; Deng et al. 2023), which would result in incorrect outputs and erroneous predictions. Additionally, a more critical concern arises from libraries' underlying implementations. To enhance computational speed and facilitate memory resource management, frameworks like PyTorch (PyTorch 2025) and TensorFlow (TensorFlow 2025) rely on memory-unsafe languages (i.e., C/C++) for implementing core operations, which increases the likelihood of vulnerabilities such as memory overflow and null pointer dereference (Filus and Domanska 2023). Attackers leveraging these flaws could corrupt memory structures, exfiltrate sensitive data, or trigger application crashes (Christou et al. 2023). Such breaches ultimately pose significant risks to real-world

*Correspondence:

Yue Wu

wuyue@sjtu.edu.cn

¹ School of Computer Science, Shanghai Jiao Tong University, 800 Dongchuan RD, Minhang District, Shanghai 200240, China

deployments, compromising both system reliability and data integrity (Force 2017).

Based on the remarkable capabilities demonstrated by large language models (LLMs) in code generation (Huang et al. 2023; Islam et al. 2024; Li et al. 2023; Zhou et al. 2025; Lin et al. 2024; Fakhoury et al. 2024), TitanFuzz (Deng et al. 2023) introduced the novel paradigm of leveraging LLMs to generate DL testing programs. Subsequently, FuzzGPT (Deng et al. 2024) enhanced this framework by fine-tuning the models with bug-triggering programs to generate more edge cases. However, these methods suffer from critical limitations. The primary issue is that the generated DL programs are one-time artifacts that cannot be reused in long-term fuzzing campaigns. Within each synthesized program, both API inputs and behavioral patterns remain statically determined. This forces persistent LLM queries for new variants during extended testing, resulting in substantial computational overhead and token costs.

Compared to TitanFuzz (Deng et al. 2023) and FuzzGPT (Deng et al. 2024), regular fuzzing engines can automatically generate diverse test inputs without relying on interactions with LLM servers. However, they typically require manually constructed fuzz drivers to guide API usage and perform data flow and control flow validation. This manual process becomes particularly burdensome when applied to DL libraries, which contain a large number of APIs with complex behaviors and interdependencies.

To address the aforementioned issues, we propose FD-FACTORY, a fully automated framework that leverages LLMs to generate fuzz drivers for DL API testing. Inspired by the modular design of industrial production lines, FD-FACTORY decomposes the generation process into eight distinct stages: *Preparation, Initial Fuzz Driver Generation, Early Stop Checks, Verification, Issue Diagnosis, Decision Making, Repair Loop, and Deployment*. Each stage is handled by dedicated agents or tools to enhance construction efficiency and improve the quality of generated fuzz drivers.

FD-FACTORY incorporates several key innovations across its modular pipeline. Unlike prior approaches (e.g., TitanFuzz (Deng et al. 2023) and FuzzGPT (Deng et al. 2024)) that generate one-time and static testing programs, FD-FACTORY produces reusable fuzz drivers that can be directly integrated with industrial fuzzing engines for long-term and continuous testing. Besides, FD-FACTORY introduces an automated constraint analysis mechanism that leverages LLM to parse API documentation and usage examples, extracting invocation

rules and inter-parameter constraints. Different from prior methods (e.g., DocTer Xie et al. 2022) which require manual annotation, our approach fully automates constraint inference using sophisticated prompt templates combined with the semantic understanding capability of LLMs. During fuzz driver generation, we decouple the program into three sub-functions, i.e., target function, data constructor, and entry point, thereby reducing task complexity for LLMs and improving the validity of generated fuzz drivers. Additionally, unlike prior works (e.g., TitanFuzz (Deng et al. 2023), FuzzGPT (Deng et al. 2024), and LLM4FDG (Zhang et al. 2024)) that rely on manual validation, FD-FACTORY integrates static analysis, dynamic fuzzing, and issue diagnosis agents into a complete automated verification pipeline. In particular, the early stop module proactively filters out structurally invalid or hallucinated outputs, mitigating efficiency loss due to LLM errors and forgetting. In practice, FD-FACTORY also adopts certain established components, such as the generate-repair paradigm (Nunez et al. 2024), the industrial fuzzing engine Atheris (Google 2025), and the static analysis tool Ruff (Ruff 2025), to ensure practicality and robustness.

Experimental results show that FD-FACTORY achieves success rates of 73.67% on PyTorch and 65.33% on TensorFlow, representing improvements of 34.66% to 54.66% over existing approaches. To further evaluate the effectiveness of the generated fuzz drivers, we conducted 30-minute fuzzing for each fuzz driver. FD-FACTORY achieves total coverage of 308,351 lines on PyTorch and 528,427 lines on TensorFlow, substantially exceeding the coverage reported by prior methods. Besides, it uncovered two known vulnerabilities in PyTorch, highlighting the practical potential of our approach.

In summary, the contributions of our work are listed as follows.

- We propose FD-FACTORY, a fully automated framework that leverages LLMs to generate fuzz drivers for DL API testing.
- Inspired by the modular design of industrial production lines, FD-FACTORY decomposes the generation process into eight distinct stages to enhance the validity of generated fuzz drivers.
- FD-FACTORY integrates an API constraint analysis module that leverages LLMs to extract usage constraints from external sources such as API documentation to enhance the effectiveness of subsequent fuzz driver generation.

- FD-FACTORY restructures the initial fuzz driver generation process into three stages, i.e., target function generation, input data construction, and final fuzz driver synthesis, which improves the success rate of generating valid fuzz drivers.
- FD-FACTORY introduces an automated validation workflow combining early stop checks, static analysis, and dynamic fuzzing to ensure the correctness of generated fuzz drivers. It further incorporates an *Issue Diagnosis* module for error localization and repair suggestion, with decisions guided by a dedicated *Decision Making* module.
- Experimental results show that FD-FACTORY achieves success rates of 73.67% on PyTorch and 65.33% on TensorFlow, outperforming existing approaches by 34.66% to 54.66%. It also attains a total coverage of 308,351 lines on PyTorch and 528,427 lines on TensorFlow, significantly surpassing the coverage reported by prior methods. These results demonstrate the effectiveness of FD-FACTORY in leveraging LLMs to generate valid fuzz drivers for DL library testing.

The remainder of this paper is organized as follows. Section "Background" provides background information and reviews related work. Section "Approach" describes the proposed approach in detail. Section "Case study" presents a case study. Section "Evaluation" outlines the experimental setup and presents the evaluation results. Finally, section "Conclusion" concludes the paper.

Background

Bug detection for DL libraries

Bug detection methods for DL libraries are usually categorized into two paradigms: model-level detection (e.g., CRADLE (Pham et al. 2019), Audee (Guo et al. 2020), LEMON (Wang et al. 2020), Muffin (Gu et al. 2022), and Gandalf (Liu et al. 2024)) and API-level detection (e.g., DocTer (Xie et al. 2022), IvySyn (Christou et al. 2023), FreeFuzz (Wei et al. 2022), DeepREL (Deng et al. 2022), and VFuzz (Yang et al. 2023)). The former requires complete neural network models as test subjects, while the latter directly targets individual library APIs.

However, Deng et al. (2023) indicated that these methods failed to generate arbitrary tests with diverse API sequences and usage patterns. These limitations have catalyzed the exploration of LLMs as a paradigm-shifting testing tool (Deng et al. 2023). Unlike previous approaches that rely on collecting existing models (Pham

et al. 2019; Guo et al. 2020; Wang et al. 2020) and API usage code snippets (Wei et al. 2022; Deng et al. 2022) followed by simplistic constraint analysis (Xie et al. 2022; Wei et al. 2022) and mutation (Guo et al. 2020; Wang et al. 2020; Wei et al. 2022), LLM-based methods inherently capture latent semantic relationships between APIs from their pretraining corpora. This enables the synthesis of syntactically valid yet semantically unconventional API call sequences. Although TitanFuzz (Deng et al. 2023) and FuzzGPT (Deng et al. 2024) demonstrate promising results in DL testing, the following limitations have significantly reduced their effectiveness and scalability in practical fuzzing.

- **One-time static programs.** Both approaches generate test programs with hardcoded inputs and fixed API invocations. These programs are non-reusable and incompatible with iterative or long-running fuzzing processes. Given the high latency and computational cost of LLM queries, such a design becomes prohibitively expensive for large-scale or long-term testing.
- **Low generation success rate.** LLM generated programs may contain syntax errors, semantic mistakes, and model hallucinations (Liu et al. 2024; Huang et al. 2024). The success rate of generating effective test programs with TitanFuzz and FuzzGPT is only around 30%, which presents significant waste in computational resources.
- **Lack of reuse and repair.** Generated programs are treated as throwaway artifacts without mechanisms for reuse, validation, or refinement. Without repair capabilities, invalid outputs from LLMs are discarded entirely, rendering many LLM queries unproductive.

LLM-based fuzz driver generation

While prior research on LLM4FDG (Zhang et al. 2024) demonstrated the feasibility of using LLMs to generate fuzz drivers for C/C++ APIs, its simplistic architecture presents several shortcomings. Most notably, it fails to support Python APIs and suffers from low valid code generation rates. Beyond these, fuzz driver generation for DL library APIs presents more challenges. In LLM4FDG, only 86 APIs that accept input with simple structures such as files or network packages were tested. The authors employed a semi-automated fuzz driver verification strategy with manually crafted validation rules for each API. However, APIs of the DL libraries are much more diverse and numerous. Thus, manually designing validation rules

for each API would be impractical, necessitating an automated validation approach. Furthermore, the diverse contextual and input requirements across APIs necessitate a more in-depth analysis of API constraints. For instance, DL library APIs typically accept parameters like tensors and integers, requiring specific data type conversions and validations in fuzz drivers (e.g., tensor for input, int for shape). However, Python's dynamic typing performs type checking at runtime without explicit type declarations, making API parameter types ambiguous. Moreover, DL APIs often contain multiple parameters with inter-parameter constraints (e.g., tensor shapes) and may require specific invocation contexts. LLM4FDG lacks corresponding constraint analysis, resulting in extremely low success rates for valid code generation.

Our design is inspired by TitanFuzz and FuzzGPT, which first demonstrated the feasibility of using LLMs to generate DL-related testing programs. We also draw from

LLM4FDG, which adopts a generation–repair loop to generate more sustainable fuzz driver programs. However, unlike these works, FD-FACTORY incorporates automated constraint analysis, supports both Python and native C/C++ coverage, and enables fully automated validation. These advances overcome prior limitations, including lack of reusability, absence of automated validation, and low generation success rates.

Fuzz driver for DL library testing

The standard structure of a fuzz driver $\mathcal{F}$ for testing DL library APIs is presented in Listing 1, which can be abstracted into two major components: the fuzz target $\mathcal{T}$ and auxiliary code $\mathcal{A}$:

$$\mathcal{F} = \langle \mathcal{T}, \mathcal{A} \rangle.$$

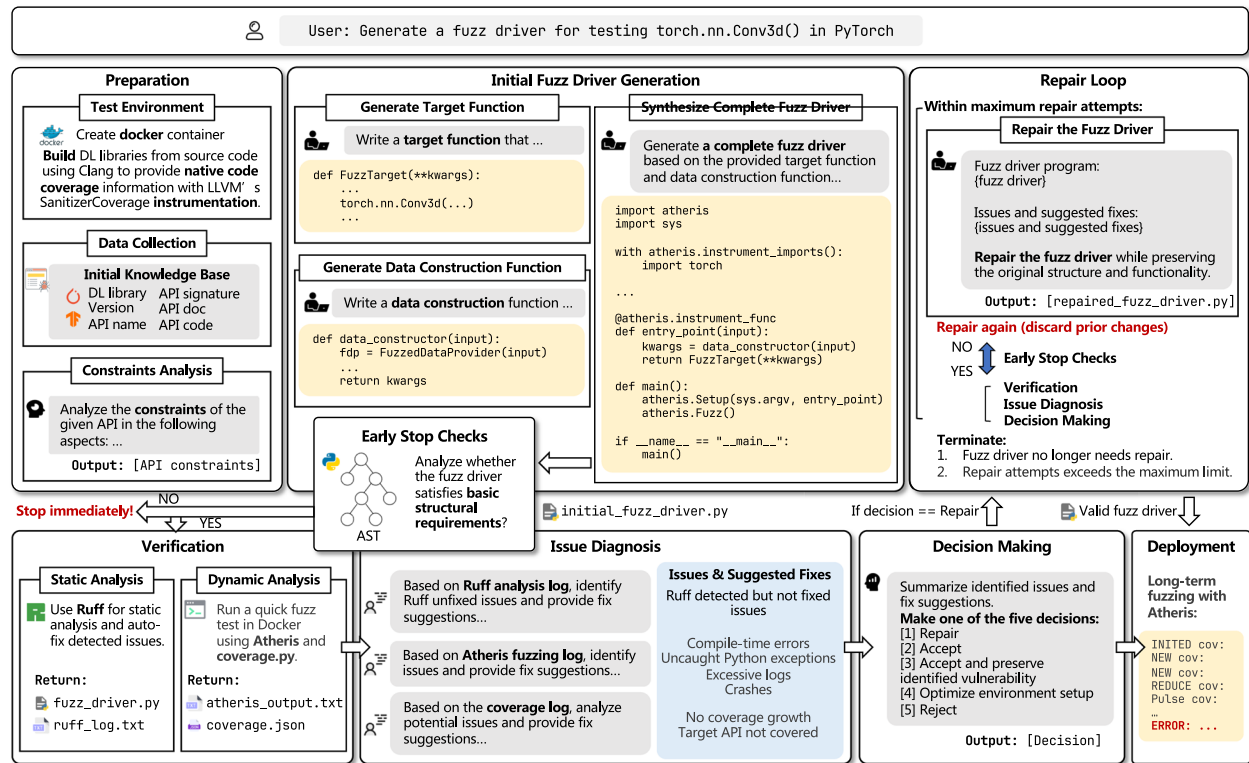


Fig. 1 Overview of FD-FACTORY

```

1 # Auxiliary code: imports and instrumentation
2 import atheris
3 import sys
4 from atheris import FuzzedDataProvider
5
6 with atheris.instrument_imports():
7     import torch
8
9 @atheris.instrument_func
10 def TestOneInput(input): # Critical: Fuzz Target
11     params = DataTransform(input)
12     Validate(params)
13
14     try:
15         # (Opt.) Pre-conditions
16         Target_API(params) # API invocation
17         # (Opt.) Post-conditions and API interactions
18
19     except Exceptions as e:
20         # (Opt.) Edge-case handler
21
22     finally:
23         # (Opt.) Resoucre cleanup
24         return
25
26 # Auxiliary code: Fuzz configuration and bootstrap
27 def main():
28     atheris.Setup(sys.argv, TestOneInput)
29     atheris.Fuzz()
30
31 if __name__ == "__main__":
32     main()
33

```

Listing 1: Demonstration of the basic structure for DL testing fuzz drivers

```

1 # Auxiliary code: imports and instrumentation
2 import atheris
3 import sys
4 from atheris import FuzzedDataProvider
5
6 with atheris.instrument_imports():
7     import torch
8
9 @atheris.instrument_func
10 def TestOneInput(input): # Critical: Fuzz Target
11     params = DataTransform(input)
12     Validate(params)
13
14     try:
15         # (Opt.) Pre-conditions
16         Target_API(params) # API invocation
17         # (Opt.) Post-conditions and API interactions
18
19     except Exceptions as e:
20         # (Opt.) Edge-case handler
21
22     finally:
23         # (Opt.) Resoucre cleanup
24         return
25
26 # Auxiliary code: Fuzz configuration and bootstrap
27 def main():
28     atheris.Setup(sys.argv, TestOneInput)
29     atheris.Fuzz()
30
31 if __name__ == "__main__":
32     main()
33

```

Listing 1: Demonstration of the basic structure for DL testing fuzz drivers

The **fuzz target** $\mathcal{T}$ represents the core test execution logic, comprising:

$$\mathcal{T} = \left\langle \begin{array}{l} \text{data transform, validations,} \\ \text{API invocations, pre/post-conditions,} \\ \text{edge-case handlers, cleanup} \end{array} \right\rangle,$$

where raw fuzzer inputs are transformed and validated to construct required API parameters, target APIs are invoked, and optional pre-/post-conditions are applied. Besides, it might include additional APIs to test vulnerabilities during API interactions. In some cases, edge case handlers like try-except blocks or if statements are involved to perform semantic oracle checks on execution results or filter out non-vulnerability-related exceptions.

Resource management and cleanup operations are also required to ensure system stability during extended fuzzing campaigns.

The **auxiliary code** $\mathcal{A}$ supports the fuzz target and ensures executable completeness, comprising:

$$\mathcal{A} = \langle \text{imports, instrumentation, configuration, bootstrap} \rangle,$$

where dependencies related to DL libraries (e.g., PyTorch, TensorFlow) must be imported at the beginning of the fuzz driver program to enable DL operations. In addition, the fuzz driver typically includes instrumentation statements for comprehensive code coverage tracking across Python and native C/C++ components. In the end, a `main()` function is usually involved to configure the fuzzing engine and bootstrap execution.

Approach

Overview of the framework

In this paper, we propose FD-FACTORY, a fully automated framework for generating fuzz drivers to test DL libraries. Inspired by the modular design of industrial production lines, FD-FACTORY decomposes the generation process into eight distinct stages, as illustrated in Fig. 1.

The workflow begins with the *Preparation* stage. Given a target DL library and API, this stage configures the testing environment, collects API-related data, and analyzes API usage constraints (section "Constraint analysis") to avoid generating invalid inputs or incorrect usage patterns in later stages. Next, the *Initial Fuzz Driver Generation* stage (section "Initial fuzz driver generation") constructs the first version of the fuzz driver by decomposing the generation process into three subtasks. Then, the *Verification* stage (section "Verification") tries to prove the validity of generated fuzz drivers through early stop checks, static analysis, and dynamic analysis. Based on the verification results, the *Issue Diagnosis* stage (section "Issue diagnosis") identifies potential flaws in the fuzz driver and provides repair suggestions. The *Decision Making* (section "Decision making") stage then aggregates all prior information to determine whether the fuzz driver requires further refinement and guides the subsequent workflow. Failed fuzz drivers enter the *Repair Loop* (section "Repair loop") for iterative repairs until success or timeout. Ultimately, valid fuzz drivers proceed to the *Deployment* stage for extended fuzzing.

Constraint analysis

The generation of fuzz drivers requires LLMs to possess deep understanding of the target API's usage patterns and constraints. Therefore, we incorporate a constraint analysis agent α that extracts essential API usage constraints

$\mathcal{C}$ by parsing the data collected by a web crawler κ in the preparation stage.

$$\kappa : \text{api} \rightarrow (\text{doc}, \text{code}), \quad \alpha : (\text{doc}, \text{code}) \rightarrow \mathcal{C}.$$

Specifically, the agent analyzes the documentation and usage code of the target API. The documentation includes the API signature, functional description, supported data types, parameter constraints, and inter-parameter dependencies. The usage code involves known API invocation patterns and provides demonstrations of valid API behaviors. These resources have covered the constraints that must be followed during API invocation. Thus, the agent analyzes them to extract API usage limitations.

The generated constraint analysis report $\mathcal{C}$ can be defined as:

$$\mathcal{C} : (\mathcal{P}, \mathcal{I}, \mathcal{G}, \mathcal{O}, \mathcal{D}) = \alpha \star \kappa,$$

where the operator $\star$ denotes the interaction between agents or tools. $\mathcal{P}$ represents the parameter specifications, including data structures (e.g., tensor, ndarray, list, tuple, dict, and set), types (e.g., int32, float64, str, and bool), value ranges, dimensional requirements (for tensors), and default values. $\mathcal{I}$ captures inter-parameter constraints such as dependencies, mutual exclusions, and conditional requirements. $\mathcal{G}$ provides contrastive valid/invalid examples for illustrating concrete constraint boundaries. $\mathcal{O}$ describes the output specifications inferred from the API. $\mathcal{D}$ summarizes contextual dependencies and required imports.

Initial fuzz driver generation

Once the preparation is done, the system begins its first attempt to generate the fuzz driver. However, the empirical study in Fig. 6 reveals that fuzz drivers directly generated by LLMs exhibit significant deficiencies in data construction, validation, and error handling. This is because constructing a complete fuzz driver entails numerous intricate steps and logic. To address these limitations, we propose a decomposition of the standard fuzz driver by splitting the fuzz target $\mathcal{T}$ into three distinct functions:

$$\text{Decouple}(\mathcal{T}) = \langle \text{data_constructor}, \\ \text{FuzzTarget}, \text{entry_point} \rangle.$$

Each component is assigned to a dedicated LLM agent for implementation. The finer-grained task decomposition enables different LLM agents to focus on specific subtasks, thereby enhancing the effectiveness of LLM-generated fuzz drivers.

Generate target function

The system first incorporates a coding agent β to generate a standalone target function `FuzzTarget(**kwargs)`, which focuses exclusively on designing appropriate API call sequences.

$$\beta : (\text{api}, \mathcal{C}) \rightarrow \text{FuzzTarget},$$

$$\begin{aligned} \text{FuzzTarget} : (\text{kwargs}, \text{pre/post}, \text{APIs}, \text{cleanup}, \text{terminate}) \\ = \beta \star \alpha. \end{aligned}$$

The agent constructs the `FuzzTarget(**kwargs)` function by strictly adhering to the previously derived constraint report $\mathcal{C}$ from agent α . In addition to the target API, the function should include necessary pre-conditions, post-conditions, resource cleanup, and proper termination. Optionally, additional API calls may be incorporated to test potential vulnerabilities during API interactions. All external inputs and parameters involved in the API sequence must be obtained exclusively through `kwargs` to eliminate the need for data construction and validation during the design phase.

Generate data construction function

To produce and validate all parameter data required by `FuzzTarget`, the system subsequently generates a `data_constructor(input)` function with agent γ .

$$\gamma : (\text{api}, \mathcal{C}, \text{FuzzTarget}) \rightarrow \text{data_constructor}$$

$$\text{data_constructor} : (\text{generation}, \text{validation}) = \gamma \star (\beta \cup \alpha)$$

Agent γ accepts the target function from agent β and the constraint report from agent α . It first identifies all required parameters and their constraints. Then, it constructs data using mutated input from the fuzzing engine for randomized parameters while assigning fixed values to deterministic ones. Subsequently, the function validates all generated data against known constraints to prevent invalid inputs that could lead to premature termination of the fuzz driver. Finally, all parameter data is assembled into a `kwargs` dictionary for `FuzzTarget` consumption.

Synthesize complete fuzz driver

The fuzz driver synthesis agent δ integrates `FuzzTarget` with `data_constructor` while completing all missing elements to generate a complete fuzz driver $\mathcal{F}$.

$$\delta : (\text{api}, \mathcal{C}, \text{FuzzTarget}, \text{data_constructor}) \rightarrow \mathcal{F},$$

$$\begin{aligned} \mathcal{F} &= \langle \text{import}, \text{instrument}, \text{data_constructor}, \text{FuzzTarget}, \text{entry_point}, \text{main} \rangle \\ &= \delta \star (\gamma \cup \beta). \end{aligned}$$

Specifically, it requires importing all necessary dependencies and instrumenting critical modules related to the DL libraries. Additionally, it needs to implement an `entry_point(input)` function that orchestrates the `data_constructor` and `FuzzTarget` operations, acting as the primary interface for fuzzer-generated inputs. Finally, the `main()` function configures the fuzzing engine and launches the execution.

Verification

Early stop checks

Given the potential for model hallucinations and the inherent uncertainty in LLM outputs (Zhao et al. 2023), it is imperative for the system to implement a series of validations on the generated fuzz driver programs to ensure their correctness. Empirical studies indicate occasional critical failures where the LLM misinterprets our design specifications. These deviations may manifest in various forms, including: (1) Only return code structure examples from the prompt instead of providing concrete implementations; (2) Omission of essential imports (e.g., `torch`, `tf`); (3) Absence of required functions; (4) Incorrect function signatures; (5) Lack of module and function instrumentation; (6) Incomplete fuzzer initialization; and (7) Python syntax errors. Such issues violate fundamental design requirements, thereby reducing the likelihood of successful repairs in the future. Thus, the best choice is to proactively detect and terminate the generation process for these fuzz drivers or revert to a previously validated state for redesign.

To address the issues, we propose an *early stop checks* strategy for rapid preliminary screening of fuzz driver programs, aimed at filtering out fundamental flaws. The strategy can be described as follows:

$$\text{EarlyStop}(\mathcal{F}) = \begin{cases} \text{reject}, & \text{if } \exists e \in \text{Check}(\mathcal{F}) \text{ s.t. } e \text{ is a fundamental flaw,} \\ \text{accept}, & \text{otherwise.} \end{cases}$$

$$\text{Check}(\mathcal{F}) = (\text{SyntaxCheck}(\mathcal{F}), \text{ASTTraversal}(\mathcal{F})).$$

We developed an automated detection tool based on abstract syntax trees (AST). The tool first converts the fuzz driver program into an AST and identifies basic Python syntax errors based on the conversion status. It then performs top-down traversal of AST nodes to verify whether the fuzz driver meets the basic requirements. Any inconsistencies that indicate fundamental flaws will immediately terminate the generation process if they occur in the initial round of fuzz driver generation.

(1) Detected Errors: <pre>fuzz_driver.py:7:21: F401 [*] `numpy` imported but unused 5 with atheris.instrument_imports(): 6 import torch 7 import numpy as np ^^^ F401 = help: Remove unused import: `numpy` code_after_ruff_fix.py:37:18: F821 Undefined name `alpha` 35 "input": input, 36 "other": other, 37 "alpha": alpha, ^^^^^ F821 38 "out": out, 39 }</pre> Found 2 errors. [*] 1 fixable with the `--fix` option.	(2) Proposed fixes (diff): <pre>--- fuzz_driver.py +++ fuzz_driver.py @@ -4,7 +4,6 @@ with atheris.instrument_imports(): import torch - import numpy as np Would fix 1 error. (3) After auto-fix: <pre>fuzz_driver.py:36:18: F821 Undefined name `alpha` 34 "input": input, 35 "other": other, 36 "alpha": alpha, ^^^^^ F821 37 "out": out, 38 }</pre> Found 2 errors (1 fixed, 1 remaining). </pre>
--	--

Fig. 2 Example of the static analysis logs generated by Ruff

Otherwise, the fuzz driver will revert to its previous stable state for redesign or repair iterations.

Static analysis

The system performs static analysis with Ruff (Ruff 2025) to detect issues in LLM-generated fuzz drivers. The tool is employed to detect and auto-fix issues while generating comprehensive diagnostic reports. A demonstration of the analysis log is presented in Fig. 2, which identified an unused import error (auto-fixed) and an undefined variable error (not fixed).

The output of static analysis can be described as follows:

$$\text{Ruff}(\mathcal{F}) = (\mathcal{E}, \mathcal{R}, \mathcal{F}'),$$

where $\mathcal{E}$ represents the identified errors, $\mathcal{R}$ represents repair suggestions, and $\mathcal{F}'$ represents auto-fixed fuzz driver program. These information assist subsequent repair modules in addressing problems that cannot be automatically corrected. Additionally, since the tool can detect coding style issues, it is also utilized to improve code quality.

Dynamic analysis

To verify the practical performance of fuzz drivers, the system executes them within a Docker container for a short round of fuzz testing. The process can be formulated as follows:

$$\text{RuntimeVerify}(\mathcal{F}) = (\mathcal{E}, \mathcal{L}, \mathcal{V}),$$

where $\mathcal{E}$ represents identified runtime errors, $\mathcal{L}$ represents execution logs, and $\mathcal{V}$ represents code coverage changes. The runtime errors $\mathcal{E}$ often arise from improper data transformations, insufficient data validation, incorrect API interaction patterns, or inadequate exception handling. The output logs and traceback information $\mathcal{L}$ generated during execution provide critical insights for future identifying and resolving these issues. Besides, we monitor the code coverage growth $\mathcal{V}$ throughout the fuzzing process. By examining whether coverage increases and whether the target APIs are exercised, we can assess the fuzz driver's ability to successfully invoke the target APIs and explore their internal paths. This, in turn, serves as an indicator of the fuzz driver's capability to detect potential vulnerabilities.

Issue diagnosis

The system then separately analyzes the Ruff analysis logs, the dynamic execution output of the fuzz driver, and the coverage reports with a issue diagnosis agent ϵ to identify potential issues and propose corresponding repair suggestions.

$$\epsilon : (\text{Ruff}, \text{RuntimeVerify}) \rightarrow \mathcal{X},$$

$$\mathcal{X} = \langle \text{Issue}, \text{Repair} \rangle$$

$$= \epsilon \star (\text{Ruff} \cup \text{RuntimeVerify}),$$

where $\mathcal{X}$ represents the issue diagnosis report.

For the Ruff analysis logs, the system first identifies issues that have been detected but cannot be automatically resolved, such as the undefined variable illustrated in Fig. 2. For each issue, the agent is tasked with providing a comprehensive problem description, pinpointing the exact code location, analyzing the root cause, and proposing a repair suggestion while preserving the original structure of the fuzz driver.

For the dynamic execution results, the system must analyze the reported errors and runtime logs to identify potential issues. This encompasses compilation errors reported by the Python interpreter, emitted User-Warnings, uncaught Python exceptions, unknown crashes, and frequently recurring log messages indicating either redundant logging behavior or persistent unresolved issues in the fuzz driver. Typically, such problems originate from design flaws in the fuzz driver itself. However, occasionally they may reflect bugs in the target API or misconfigurations within the fuzzing environment. The agent must classify each detected issue by type, provide a clear description, pinpoint the precise location of the error, and offer detailed repair recommendations.

For the coverage reports, the system must assess two critical factors: whether code coverage increases and whether the target API is effectively invoked. Both metrics are essential indicators of the fuzz driver's efficacy. A failure to invoke the target API typically signifies that fuzzing data is not effectively delivered to the intended function, often due to flaws in data construction, inadequate validation mechanisms, or API invocation errors. Similarly, stagnant code coverage throughout testing highlights the fuzz driver's inability to explore diverse execution paths within the API, indicating a need for enhancement. In such scenarios, the agent is tasked with identifying the precise issue within the fuzz driver, thoroughly analyzing the root cause, and provide repair recommendations.

Decision making

Based on the issue diagnosis report, the decision making agent ζ determines the most appropriate action for the next step.

$$\zeta : (\mathcal{F}, \mathcal{X}) \rightarrow \text{Decision}$$

$$\text{Decision} = \{\text{Repair}, \text{Accept}, \text{Reject}\} = \zeta \star \epsilon$$

Fuzz Driver: torch.nn.AdaptiveLogSoftmaxWithLoss

```
@atheris.instrument_func
def data_constructor(input) -> dict:
    fdp = FuzzedDataProvider(input)
    ...
    div_value = fdp.ConsumeFloatInRange(0.0, 10.0)
    ...

    kwargs = {
        "in_features": in_features,
        "n_classes": n_classes,
        "cutoffs": cutoffs,
        "div_value": div_value,
        "head_bias": head_bias,
    }
    return kwargs

@atheris.instrument_func
def FuzzTarget(**kwargs):
    in_features = kwargs.get("in_features")
    n_classes = kwargs.get("n_classes")
    cutoffs = kwargs.get("cutoffs")
    div_value = kwargs.get("div_value", 4.0)
    head_bias = kwargs.get("head_bias", False)

    model = torch.nn.AdaptiveLogSoftmaxWithLoss(
        in_features, n_classes, cutoffs, head_bias
    )
    ...
    Trigger the crash while div_value equals zero
```

Crash: Division by zero

```
== Uncaught Python exception: ==
ZeroDivisionError: float floor division by zero
Traceback (most recent call last):
  File "/home/FD-FACTORY/data/crash/.fuzz_driver.py", line 56, in
    entry_point
    return FuzzTarget(**kwargs)
  File "/home/FD-FACTORY/data/crash/.fuzz_driver.py", line 42, in FuzzTarget
    model = torch.nn.AdaptiveLogSoftmaxWithLoss(
  File "/usr/local/lib/python3.9/dist-packages/torch/nn/modules/adaptive.py",
    line 153, in __init__
        hsz = int(self.in_features // (self.div_value** (i + 1)))
ZeroDivisionError: float floor division by zero
```

Vulnerability

Fig. 3 Case study: division by zero vulnerability triggered by the fuzz driver of torch.nn.AdaptiveLogSoftmaxWithLoss

If the system concludes that the current fuzz driver exhibits design flaws, it selects the **Repair** option and specifies the problems that require attention along with corresponding repair suggestions. If the fuzz driver is deemed effective and free of critical flaws, the system selects **Accept**, indicating that the fuzz driver will be retained without modification. Finally, if the current fuzz

driver is found to have too many flaws or is impractical to repair, the system may decide to **Reject** it and terminate the ongoing repair process.

Repair loop

If the decision is **Repair**, the system enters an iterative repair loop to progressively refine the fuzz driver. This loop continues until either (1) the decision maker subsequently determines that no further repair is needed, or (2) a predefined maximum number of allowed repair iterations is reached, at which point the process terminates automatically. During this process, agent η is assigned for program repair.

$$\eta: (\mathcal{F}_i, \mathcal{X}_i) \rightarrow \mathcal{F}_{i+1}, \quad \mathcal{F}_{i+1} = \eta \star \zeta \star \epsilon,$$

$$\text{RepairLoop}(\mathcal{F}_0) = \{\mathcal{F}_1, \mathcal{F}_2, \dots, \mathcal{F}_k\},$$

$$\text{Termination: } \begin{cases} (1) \text{ Decision} = \text{Accept}, \\ (2) i \geq i_{\max}. \\ (3) \text{ EarlyStop fails} \Rightarrow \mathcal{F}_{i+1} \leftarrow \mathcal{F}_i \text{ (revert to stable state).} \end{cases}$$

This agent receives the list of issues and suggested repairs ($\mathcal{X}_i$) summarized by the decision maker and performs actual code-level optimizations. During this process, it is required to preserve the structural design of the original fuzz driver, avoiding the LLM forgetting the original objectives for fuzz drive design. Once the agent completes the repair, the updated fuzz driver undergoes a new round of validation, following the same procedures, i.e., early stop checks, verification, issue diagnosis, and decision making. However, in contrast to the initial round of validation, failure to pass early stop checks in the repair loop does not result in immediate termination. Instead, the system reverts the fuzz driver to its previous stable version and initiates another repair attempt. This cycle repeats until the decision maker chooses to stop the process or the iteration limit is reached.

Case study

To demonstrate how FD-FACTORY generates effective fuzz drivers capable of revealing real-world vulnerabilities, we provide a representative example targeting `torch.nn.AdaptiveLogSoftmaxWithLoss`. Figure 3 illustrates the essential parts of the fuzz driver, along with the vulnerability it uncovered during execution.

At beginning, the `data_constructor` function employs the `ConsumeFloatInRange` method from the `FuzzedDataProvider` class to produce a random floating-point number between 0.0 and 10.0 for the `div_value` parameter. This value is then incorporated into a `kwargs` dictionary and passed to the `FuzzTarget` function, where it is correctly parsed and mapped to the corresponding argument of the target API. During execution, the `div_value` parameter is propagated into the underlying implementation file `adaptive.py`, where it is used in the following computation: `hsz = int(self.in_features // (self.div_value ** (i + 1)))`. As this expression lacks input validation, a division-by-zero exception is raised when `div_value` equals 0.0, leading to a runtime crash. The fuzzing engine captures this crash and saves the corresponding input to facilitate reproducibility.

Evaluation

Research questions

Our experiments address the following research

questions:

- RQ1: How does FD-FACTORY compare with the SOTA approaches?
- RQ2: How do the key components of FD-FACTORY contribute to its effectiveness?
- RQ3: What are the advantages and disadvantages of using LLMs to generate fuzz drivers for DL library testing?

Experimental setup

DL libraries: Following prior works such as TitanFuzz (Deng et al. 2023) and FuzzGPT (Deng et al. 2024), our evaluation used PyTorch 1.12 and TensorFlow 2.10 for consistency and comparability. Although newer versions exist, API-level differences among recent releases are minor and mainly involve incremental updates (e.g., performance optimizations or parameter extensions), which do not substantially alter the behavior of the tested APIs. Therefore, the experimental results remain representative across versions.

Data collection: We extracted 1,463 APIs from PyTorch v1.12 and 2,205 APIs from TensorFlow v2.10. To ensure the quality of our dataset, we excluded APIs that are experimental, deprecated, auxiliary, or not accept external inputs.

LLM: The optimal LLM settings in LLM4FDG (Zhang et al. 2024) are adopted in our experiments, i.e., `gpt-4-0613` with a temperature of 0.5 and the default `top_p` value.

Prompt design strategies: The prompt templates used in this paper is presented in Appendix A. The design of our prompts follows several key strategies. First, the *system prompt* defines the agent roles, provides a high-level overview of the task, and pinpoints the output format. The *user prompt* supplies essential context, outlines precise task requirements, and specifies all necessary constraints. For code generation tasks, it provides suggested code structures. Step-by-step instructions are also provided to align with the CoT paradigm (Wei et al. 2022), along with clearly defined output formats. These design choices aim to improve the response quality, ensure

Table 1 Comparison of success rates in LLM-based program generation for deep learning library Fuzzing

Framework	One-time Program ¹		Reusable Fuzz Drivers ²	
	TitanFuzz (%)	FuzzGPT (%)	LLM4FDG-DL (%)	FD-FACTORY (%)
PyTorch	38.20	27.69	19.33	73.67
TensorFlow	30.67	17.41	10.67	65.33

Bold value refers to the deep learning framework used for testing. One-time Program indicates that the generated test program is a program with hard-coded inputs and fixed behavioral patterns. Reusable Fuzz Drivers means that the generated program is a reusable fuzz driver

¹ Single-use test program with hard-coded inputs and fixed behavioral patterns

² Configurable fuzz driver designed for repeated use, supporting a wide range of input variants

structural consistency, and support automated downstream analysis.

Fuzzing budget: In dynamic analysis, each fuzz driver undergoes a one-minute fuzz test to verify its validity. For fuzz drivers that are confirmed to be valid, we perform 30-minute fuzzing to evaluate the code coverage.

Maximum repair attempts and the number of repeated experiments: To limit computational overhead while maintaining effectiveness, each fuzz driver is allowed a maximum of 5 repair attempts. Besides, each API was tested across 6 independent runs to mitigate the inherent randomness of LLM's outputs. The combination is proved to be the optimal settings in LLM4FDG.

Cost control: To keep the experimental cost within a reasonable budget, we randomly selected 100 APIs, i.e., 50 from PyTorch and 50 from TensorFlow, for fuzz driver generation.

Environment: The system is implemented on Ubuntu 20.04 LTS with Docker 27.3.1 and Python 3.9.5. The computer has a 13th Gen Intel(R) Core(TM) i7-13700KF @

3.40GHz CPU and a NVIDIA GeForce RTX 4070 Ti 12 G GPU, with each verification process allocated a default of 4GB of memory and each fuzzing process allocated a default of 8GB of memory.

Baselines

TitanFuzz (Deng et al. 2023) and FuzzGPT (Deng et al. 2024) employ LLMs to generate test programs with hard-coded inputs and API patterns for DL library testing. We compare FD-FACTORY with both TitanFuzz and FuzzGPT to assess its effectiveness in producing valid test programs and achieving meaningful code coverage.

LLM4FDG (Zhang et al. 2024) is built upon a query-fix architecture to generate fuzz drivers for C/C++ programs using LLMs. However, it cannot be directly applied to DL libraries, as their developer-accessible APIs are predominantly written in Python. Furthermore, it requires adaptation to be compatible with the Python-explicit fuzzer. To address this, we extend LLM4FDG into a customized variant, LLM4FDG-DL, by modifying the original prompts to explicitly guide the generation of Python-based fuzz drivers compatible with Atheris (Google 2025), while preserving the original design philosophy. We compare FD-FACTORY with LLM4FDG-DL to demonstrate LLMs' capability in generating valid fuzz drivers for DL libraries.

In addition, we also compare our approach with other SOTA methods, i.e., Muffin (Gu et al. 2022), FreeFuzz (Wei et al. 2022), DeepREL (Deng et al. 2022), and VFuzz (Yang et al. 2023), that specifically designed for fuzzing DL library APIs to further evaluate the effectiveness of our approach in achieving high code coverage during fuzz testing.

Table 2 Experimental results comparison between LLM4FDG-DL and FD-FACTORY

Framework	Method	Results				
		Early Stop	Static	Dynamic	Repair	Valid
PyTorch	LLM4FDG-DL	— ¹	—	624	565	58/241 ²
	FD-FACTORY	23	69	685	648	221
TensorFlow	LLM4FDG-DL	—	—	673	614	32/217
	FD-FACTORY	29	71	742	697	196

Bold value refers to the deep learning framework used for testing. Method represents the compared approaches. Early Stop corresponds to the early-stop module designed in this paper. Static and Dynamic indicate the number of issues identified through static analysis and dynamic analysis in the proposed verification process. Repair denotes the number of code repair attempts, and Valid refers to the number of valid fuzz drivers generated

¹ "—" indicates the absence of a certain capability

² For LLM4FDG-DL, the number of valid fuzz drivers is presented in the format N/M , where M denotes the number of fuzz drivers considered valid by its original verification mechanism, and N represents those that pass our enhanced verification process

RQ1: How does the performance of FD-FACTORY compare with the SOTA approaches?

Validity of Generated Test Programs. We compare FD-FACTORY with TitanFuzz (Deng et al. 2023), FuzzGPT (Deng et al. 2024), and LLM4FDG-DL (Zhang et al. 2024) to evaluate its performance in generating valid test programs for fuzzing DL libraries. The average success rate is defined in Equation 1:

$$\text{Success Rate} = \frac{\# \text{Valid Test Programs}}{\# \text{Test Generation Cycles}}, \quad (1)$$

which measures the proportion of valid test programs produced over all test generation cycles. For TitanFuzz and FuzzGPT, which do not incorporate a repair mechanism, each generation cycle corresponds directly to a single generation attempt. In contrast, for LLM4FDG-DL and FD-FACTORY, a generation cycle encompasses both program generation and subsequent repair operations.

Experimental results are presented in Table 1, which demonstrate that FD-FACTORY achieves the highest success rates in both PyTorch and TensorFlow testing, reaching 73.67 and 65.33%, respectively. Compared to other approaches, it yields a success rate improvement of 35.47 to – 54.34% on PyTorch, and 34.66 to – 54.66% on TensorFlow. TitanFuzz and FuzzGPT generate code snippets with fixed inputs and predefined API usage patterns. Due to their lower code complexity and the relatively simpler generation tasks, both methods achieve higher success rates than LLM4FDG-DL. However, since neither method incorporates a code repair mechanism, their overall success rates remain consistently lower than that of FD-FACTORY. In contrast, LLM4FDG-DL produces complete fuzz drivers. Although it incorporates a repair

mechanism, its original design lacks constraint analysis on target API invocations and its semi-automated strategy makes it failed to identify program bugs while losing manually designed verification rules. This limitation makes it more prone to generation errors, leading to lower success rates in both PyTorch and TensorFlow experiments.

Repair and Validation Strategies. Since both LLM4FDG-DL and FD-FACTORY incorporate automated code repair mechanisms, we conducted a detailed comparison between the two approaches. The results are summarized in Table 2.

As LLM4FDG-DL does not implement early stop checks or static analysis, the corresponding entries are marked as “–” to indicate the absence of these capabilities. For LLM4FDG-DL, the number of valid fuzz drivers is presented in the format N/M , where M denotes the number of fuzz drivers considered valid by its original verification mechanism, and N represents those that pass our enhanced verification process. It can be observed that the lack of an automated validation process in LLM4FDG-DL results in a high number of false positives. Without the manually established validation rules, relying solely on its original coverage detection criteria to assess the validity of fuzz drivers leads to many incorrect classifications. Specifically, only 24.07% (58/241) of the fuzz drivers for PyTorch and 14.75% (32/217) for TensorFlow were truly valid under our more rigorous evaluation.

In contrast, our method integrates both static and dynamic analysis for better repair and verification. Specifically, the early stop module filters out fuzz drivers that severely deviate from expected design patterns, identifying 23 such cases in PyTorch and 29 in TensorFlow. Subsequently, static analysis detects and tries to auto-fix the errors to improve the code quality. It identified 69 problematic fuzz drivers in PyTorch and 71 in TensorFlow. Eventually, the code coverage and execution results are analyzed to identify more hidden errors. Due to the enhanced validation capabilities, FD-FACTORY conducted more repair attempts than LLM4FDG-DL across both frameworks. Consequently, it generated significantly more valid fuzz drivers, achieving a 3.81× to 6.13×

Table 3 Comparison of code coverage rates in deep learning library fuzzing

Approach	PyTorch		TensorFlow	
	Code Cov	APIs	Code Cov	APIs
FreeFuzz	15,688	468	78,548	581
DeepREL	15,794	1071	82,592	1,159
∇Fuzz	15,860	1071	89,722	1,159
Muffin	–	–	79,283	79
TitanFuzz	23,823	1329	107,685	2215
FuzzGPT-FS	35,426	1377	146,487	2309
FuzzGPT-ZS	38,284	1237	126,195	1460
FuzzGPT-FT	36,463	1223	125,832	1834
LLM4FDG-DL	86,800	50	251,293	50
FD-FACTORY	308,351	50	528,427	50

Bold value represents the compared methods. PyTorch and TensorFlow are the deep learning frameworks under test. Code Cov denotes code coverage, and APIs refers to the number of tested APIs

“–” indicates data not available or not applicable for the corresponding method

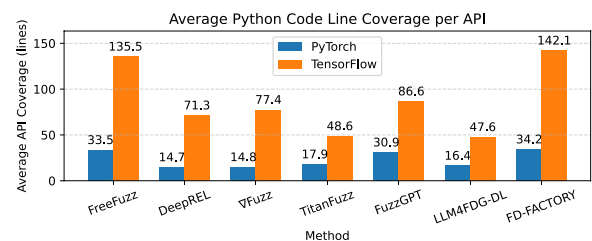


Fig. 4 Comparison of average python code line coverage per API

Table 4 Side-by-side comparison with LLM-assisted DL testing systems

Feature	TitanFuzz	FuzzGPT	LLM4FDG	FD-FACTORY
Test Reusability	One-time test program	One-time test program	Reusable fuzz drivers	Reusable DL fuzz drivers
Constraint Awareness	None	None	None	Automated constraint analysis
Generation Strategy	Monolithic generation	Monolithic generation	Generation–repair loop	Decomposed generation & repair loop
Validation Process	Manual execution	Manual execution	Human-in-the-loop validation	Automated (early stop & static & dynamic)
Coverage Support	Python only	Python only	C/C++	Python & native C/C++
Incremental Advances	–	–	–	Higher success rate, broader coverage
Fundamental Advances	–	–	Reusable fuzz driver, repair Mechanism	DL support, fuzz driver decomposition, automated constraint inference, early stop strategy, automated validation

increase compared to LLM4FDG-DL. These results demonstrate that our combined validation–repair framework substantially improves the reliability and effectiveness of fuzz driver generation for DL APIs.

Code Coverage. Table 3 presents a comparison of code coverage rates across different fuzzing approaches. In addition to evaluating LLM-based test generation methods such as TitanFuzz, FuzzGPT, and LLM4FDG-DL, we also include SOTA techniques in DL library fuzzing, including FreeFuzz, DeepREL, and VFuzz. Unlike previous methods that rely on `coverage.py` to monitor Python-level line coverage, FD-FACTORY applies source-level instrumentation during the *Preparation* stage by compiling the DL libraries from source. This enables FD-FACTORY to track both the Python code and the underlying native C/C++ code coverage of the APIs. Despite evaluating only 50 APIs and conducting 30-minute fuzzing sessions with each generated valid fuzz driver, FD-FACTORY achieves 308,351 lines of code coverage on PyTorch and 528,427 on TensorFlow. These results significantly outperform all prior approaches.

To further evaluate the comparability of different fuzzing approaches under a unified experimental setting, we additionally compared the Python code line coverage achieved by each method. Figure 4 presents the results across both PyTorch and TensorFlow frameworks. As shown in the figure, FD-FACTORY achieves a slightly higher Python-level code line coverage compared to all baseline methods in both frameworks. This improvement can be attributed to our method’s more comprehensive coverage-tracking and mutation strategies. Nevertheless, it is worth noting that, within DL frameworks, the majority of the core logic and computation is implemented in lower-level C/C++ backend code, while the Python layer mainly serves as an interface and bridge between user APIs and the underlying native implementations.

Table 5 Ablation study of FD-FACTORY on PyTorch

Configuration	Valid Programs	Coverage
Full System	221	308,351
w/o Constraints Analysis	198	264,905
w/o Decomposed Fuzz Driver	209	287,899
w/o Repair	71	69,981

Consequently, the total amount of Python code in these frameworks is relatively small, which inherently limits the extent to which Python-level coverage can be improved. Overall, these results confirm that FD-FACTORY maintains competitive and improved code coverage under comparable conditions, further demonstrating the effectiveness of our system in generating high-quality fuzz drivers for DL library testing.

Side-by-side Comparison with LLM-assisted DL Testing Systems. To better delineate the incremental and fundamental contributions of FD-FACTORY, we present a side-by-side comparison with LLM-assisted DL fuzzing systems, i.e., TitanFuzz Deng et al. (2023), FuzzGPT Deng et al. (2024), and LLM4FDG-DL Zhang et al. (2024). These systems inspired our design but also exhibit limitations that FD-FACTORY systematically addresses. The comparison is summarized in Table 4.

As shown in Table 4, TitanFuzz and FuzzGPT demonstrated the feasibility of employing LLMs to generate test programs for DL libraries, but their outputs were limited to one-time, static programs. LLM4FDG introduced the generation–repair paradigm, yet its outputs relied heavily on human validation. In contrast, FD-FACTORY introduces several fundamental advances: (i) fuzz driver reusability that supports long-term DL fuzzing campaigns with industrial engines, (ii) a decomposition-based generation workflow that reduces LLM task complexity,

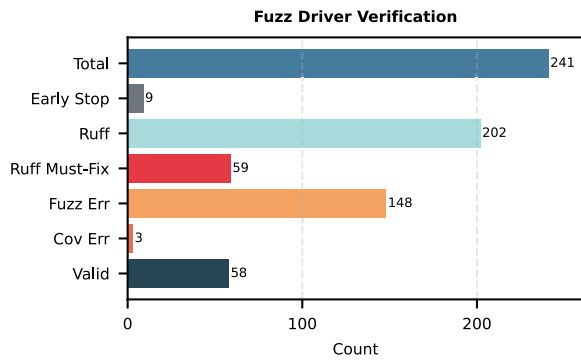


Fig. 5 Verification results of fuzz drivers generated by LLM4FDG-DL on PyTorch

Types and Rates of Problems in Invalid Fuzz Drivers

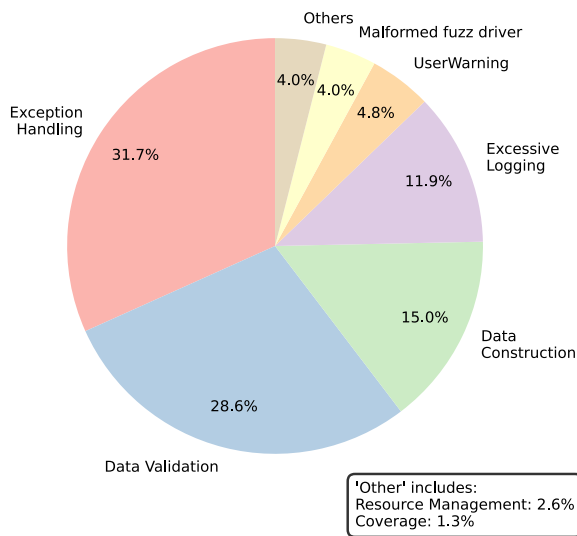


Fig. 6 The categories and proportional distribution of issues found in the fuzz drivers generated by LLM4FDG-DL for PyTorch that failed to pass the FD-FACTORY validation

(iii) automated constraint inference using LLMs without manual annotations, and (iv) a fully automated validation loop with static and dynamic analysis plus an early stop module.

In addition, FD-FACTORY achieves incremental improvements over existing methods by significantly increasing fuzz driver generation success rates and enabling broader coverage (both Python and native C/C++). Together, these features clarify the contribution boundaries of FD-FACTORY. It not only builds upon insights from prior systems but also extends them with both incremental enhancements and fundamental methodological advances.

RQ2: How do the key components of FD-FACTORY contribute to its effectiveness?

Ablation Study. The ablation study assesses the contribution of each component by comparing three reduced configurations against the full system on PyTorch. The first configuration removes the constraints analysis module, the second configuration disables the initial fuzz driver decomposition, and the third configuration disables the repair module.

As shown in Table 5, all reduced configurations generate fewer valid fuzz drivers and achieve less code coverage compared to the complete FD-FACTORY implementation. Specifically, removing the constraints analysis results in a 10.4% reduction in valid fuzz drivers and a 14.1% drop in code coverage. Disabling fuzz driver decomposition yields a 5.4% and 6.6% decrease in valid programs and coverage, respectively. The most significant impact is observed when the repair module is removed: the number of valid fuzz drivers drops by 67.9%, and the coverage decreases by 77.3%. These results highlight the critical role of the repair component in enhancing both correctness and coverage, and confirm the necessity of all three modules for maximizing fuzzing effectiveness in FD-FACTORY.

Fuzz Driver Verification and Issue Analysis. To evaluate the effectiveness of our automated validation strategies, we applied the verification framework to the fuzz drivers generated by LLM4FDG-DL, specifically targeting those that had already passed the original validation checks in the PyTorch experiments. The results, as presented in Fig. 5, revealed that out of 241 fuzz drivers, only 58 were validated by our system. Among the remaining cases, 9 fuzz drivers failed the early stop checks, and Ruff-based static analysis identified optimization opportunities in 202 fuzz drivers, with 59 containing issues that must be addressed. Our one-minute dynamic fuzzing assessment uncovered unresolved problems in 148 fuzz drivers, while the coverage-based analysis found 3 programs in which the target API was never invoked.

Subsequently, we conduct a manual analysis of the fuzz drivers that failed to pass our validation pipeline and identified eight distinct categories of issues: *malformed fuzz driver*, *data construction*, *data validation*, *UserWarning*, *exception handling*, *excessive logging*, *resource management*, and *coverage*. The distribution of these issue types is presented in Fig. 6.

The results from early stop checks suggest that LLMs, after multiple rounds of generation and repair, may forget the initial design requirements of the fuzz driver and instead focus solely on producing executable code. It constitutes 4.0% of all identified issues. This oversight is not captured by the original validation criteria in LLM4FDG-DL, which only consider a

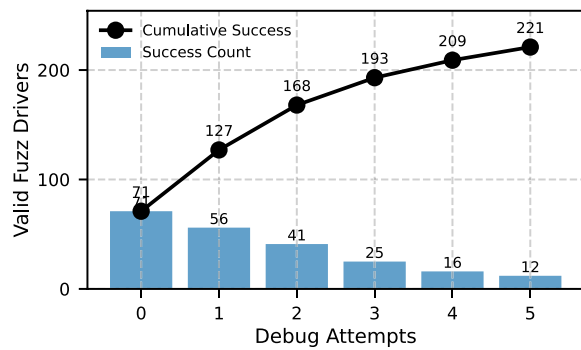


Fig. 7 Distribution of the number of valid fuzz drivers across different debug attempts in PyTorch experiments. The bar chart represents the number of valid fuzz drivers generated in a given debug round, while the line chart indicates the cumulative number of fuzz drivers up to that debug round

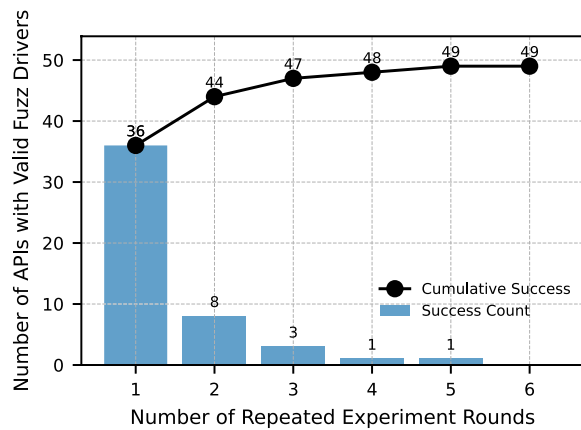


Fig. 8 Distribution of the number of APIs with valid fuzz drivers across different repeated experiment rounds in PyTorch experiments. The bar chart represents the number of APIs that succeeded for the first time in a given round, while the line chart indicates the cumulative number of successful APIs up to that round

fuzz driver effective if it compiles, terminates normally, and shows coverage growth within one minute of fuzzing. However, not all problems during execution result in crashes. Even the programs are executable or functionally correct, API misuse may still exist to affect the validity of the programs (Zhong and Wang 2024). For instance, the Python interpreter may emit `UserWarnings` (4.8%) that indicate potential flaws in the fuzz driver without terminating the fuzzing process. Moreover, improper exception (31.7%) or logging (11.9%) mechanisms in the design of the fuzz drivers may suppress meaningful error reports, redirecting critical issues to standard output or log files without raising visible exceptions. This can conceal problems related to incorrect API input construction (15.0%),

Table 6 Summary of LLM usage and overheads

Metric	Total	Per Experiment	Per Valid Fuzz Driver
API Calls	5525	18.42	25.00
Token	12.80M (\$384.08)	42.68K (\$1.28)	57.93K (\$1.74)
Time	40.35h	8.07min	10.96min

Metrics refers to the evaluation indicators. Total represents the total cost, Per Experiment indicates the average cost per experiment, and Per Valid Fuzz Driver represents the average cost of generating one valid fuzz driver. The table aims to analyze the trade-off between generation efficiency and experimental overhead when using LLMs for fuzz driver generation

inadequate validation (28.6%), and resource management (2.6%). In several cases, fuzz drivers continuously log benign program states, such as the value of a valid tensor, which adds no diagnostic value and may hinder subsequent vulnerability analysis. Additionally, the reliance of LLM4FDG-DL on coverage increase alone as the metric for effectiveness is insufficient (1.3%). Coverage can grow due to execution of the fuzz driver code itself rather than meaningful exploration of the target API's internals. Therefore, beyond observing coverage growth, it is essential to confirm whether the target API is actually exercised. Our proposed validation procedure uncovers these hidden issues by providing a more comprehensive analysis with dedicated tools and agents, thereby offering a more rigorous assessment and ensuring the reliability and quality of fuzz drivers generated by LLMs.

RQ3: What are the advantages and disadvantages of using LLMs to generate fuzz drivers for DL library testing?

LLMs demonstrate strong potential in automating fuzz driver generation for DL libraries. Trained on extensive corpora that include API documentation, DL application code, and fuzz testing scripts, LLMs are capable of inferring the intended usage patterns of DL APIs. This allows them to produce syntactically correct, semantically meaningful, and fuzzing-framework-compatible code, without relying on handcrafted invocation rules.

Nevertheless, our findings also highlight several challenges associated with this paradigm. While our framework significantly improves the generation of valid fuzz drivers, doing so requires orchestrated collaboration between multiple LLM agents and iterative code repair cycles as illustrated in Fig. 7. Given the inherent limitations in context window and reasoning capability, LLMs rarely succeed in producing a fully functional fuzz driver in a single attempt. Instead, the process must be decomposed into multiple sub-tasks, supported by external tools that execute, validate, and diagnose the generated code. As well, in many cases, it requires multiple generation attempts to get a valid fuzz driver, as illustrated in

Table 7 Repair loop overheads

Metric	Total	Per Experiment	Per Loop	Per Valid Fuzz Driver
Loops	648	2.16	–	2.93
API Calls	3169	10.56	4.89	14.34
Token	8.02M (\$240.60)	26.73K (\$0.80)	12.38K (\$0.37)	36.29K (\$1.09)
Time	25.24h	5.05min	2.34min	6.85min

Similar to the previous one, but includes Per Loop, which represents the average overhead of each repair loop, emphasizing the analysis of repair-related costs

Table 8 Cost-effectiveness comparison of system configurations

Configuration	Metrics				Valid FDs
	API Calls	Tokens (\$)	Time (h)	Repair	
Full system	5525	12.80M (\$384.1)	40.35	648	221
w/o Constraints analysis	4418	9.55M (\$286.5)	35.89	543	198
w/o Decomposition FD	4930	10.03M (\$300.9)	40.76	689	209
w/o Repair	2356	4.78M (\$143.5)	14.13	–	71

Compares the overhead of different system configurations in the ablation study. Configuration represents different configurations, Metric refers to the evaluation indicators, API Calls indicates the number of LLM queries, Tokens denotes the LLM token cost, Time represents time overhead, Repair is the number of repair attempts, and Valid FDs stands for the number of valid fuzz drivers generated

Fig. 8. This leads to substantial overhead in both time and token usage during fuzz driver generation.

LLM Usage Cost and Overhead. To further quantify the practical cost of using LLMs for fuzz driver generation, we conducted a detailed measurement of LLM API usage, token consumption, and wall-clock time for the experiments on PyTorch. The overall statistics are summarized in Table 6. FD-FACTORY made a total of 5525 API calls and consumed 12.80M tokens (\$384.08), corresponding to an average of 18.42 API calls and 42.68K tokens (\$1.28) per experiment, or 25 API calls and 57.93K tokens (\$1.74) per valid fuzz driver. The total execution time amounted to 40.35 h, averaging 8.07 min per experiment and 10.96 min per valid fuzz driver. This indicates that the overall process incurs a non-trivial computational and monetary cost. However, considering the high unit price of the OpenAI gpt-4-0613 model (\$30 per 1 M input tokens and \$60 per 1 M output tokens), the above expenditure is relatively

acceptable and reflects a reasonable trade-off between generation quality and cost efficiency.

Additionally, since a single-shot generation rarely produces a valid fuzz driver, FD-FACTORY employs iterative repair to refine partially correct fuzz drivers. Table 7 details the overhead during repair loops in the PyTorch experiments. A total of 648 repair loops were triggered during the experiments, corresponding to 2.16 loops per experiment and 2.93 loops per valid fuzz driver. Each loop involves on average 4.89 API calls, 12.38K tokens (\$0.37), and 2.34 min of wall-clock time. Although the repair mechanism accounts for roughly 63% of the total token cost, it improves the yield of valid fuzz drivers by 3.1×, demonstrating a favorable cost–benefit ratio between refinement overhead and final generation quality.

To analyze cost efficiency under different design variants, we conducted an detailed comparison of the cost for the configurations of FD-FACOTRY introduced in section "RQ2: How do the key components of FD-FACTORY contribute to its effectiveness?". The comparative results

Table 9 Normalized cost per valid fuzz driver under different configurations

Configuration	Metrics (per valid fuzz driver)				Valid FDs
	API Calls	Tokens (\$)	Time (min)	Repair	
Full System	25.00	57,930 (\$1.74)	10.95	2.93	221
w/o Constraints Analysis	22.31	48,230 (\$1.45)	10.88	2.74	198
w/o Decomposition FD	23.59	47,991 (\$1.44)	11.70	3.30	209
w/o Repair	33.18	67,359 (\$2.02)	11.94	–	71

Similar to Table 8, except that the metrics are normalized to the average cost per valid fuzz driver

are presented in Tables 8 and 9, where API calls represent the number of LLM queries, valid FDs represent valid fuzz driver programs generated during experiments.

The Full System configuration achieved the highest generation yield, producing 221 valid fuzz drivers with a per-driver cost of \$1.74. Although this is not the lowest cost per driver, it achieves the best overall balance between cost and effectiveness, given its substantially higher success rate. Disabling the constraint analysis or decomposition modules slightly reduced per-driver costs (\$1.45 and \$1.44, respectively) but also decreased the number of valid fuzz drivers (198 and 209). These results indicate that both modules contribute to improved robustness and code validity, even though they incur moderate additional cost. In contrast, removing the repair component led to a sharp drop in valid fuzz driver yield (from 221 to 71) and increased the normalized cost to \$2.02 per driver, despite fewer total API calls and tokens. This confirms that iterative repair is essential for achieving high driver quality and generation success. Overall, FD-FACTORY's complete configuration offers the best trade-off between resource cost and generation performance, demonstrating that the inclusion of all modules significantly improves fuzz driver synthesis efficiency.

Model family and deployment choice. To investigate the impact of model family choice on fuzz driver generation, we conducted additional experiments using two representative LLMs from different providers: DeepSeek (deepseek-chat-v3-0324) and xAI (Grok-Code-Fast-1). These experiments were performed on the latest PyTorch 2.9 release to ensure recency and cross-framework consistency. Due to the significantly higher API costs, we excluded OpenAI GPT-4 from these extended experiments. Across 300 fuzz driver generation experiments, the average success rates were 91.67% for deepseek-chat-v3-0324 and 84.8% for Grok-Code-Fast-1. Considering both performance and economic efficiency, deepseek-chat-v3-0324 shows the most balanced trade-off, providing the highest success rate at the lowest per-token cost.

Vulnerability discovery. Although FD-FACTORY achieved significant coverage improvement and successfully rediscovered known vulnerabilities in PyTorch, it did not uncover entirely new defects. We attribute this to multiple interrelated factors rather than an intrinsic limitation of the proposed framework.

First, the bug-finding capability of our framework is ultimately determined by the underlying fuzzing engine's data generation and mutation strategies. The focus of this study is to demonstrate the effectiveness of LLM-based fuzz driver generation, rather than optimizing the fuzzing process itself. The rediscovery of existing bugs validates

that our generated fuzz drivers are functional, accurate, and capable of triggering real defects through autonomous fuzzing execution. Whether the framework can more effectively discover vulnerabilities depends on improvements in the fuzzing engine itself, which we leave for future work.

In addition, the current experiments employ empty seed banks for all fuzzing campaigns. Since this work does not involve the design or optimization of seed inputs, the coverage and vulnerability-triggering potential are constrained by the quality of these initial seeds. To achieve deeper exploration, subsequent research could integrate DL-specific seed generation and adaptive mutation strategies, thereby enhancing the discovery of subtle or previously untested behaviors.

Due to the high computational and token costs associated with multi-agent orchestration, we limited the amount of tested APIs and fuzzing duration to maintain budget feasibility. These settings ensure a fair comparison but inevitably constrain large-scale or long-duration fuzzing campaigns, which are often required to uncover rare or previously unknown defects in mature libraries.

Moreover, the evaluated DL frameworks are highly mature and widely tested by the community, leaving few remaining undiscovered issues. The current validation focuses primarily on runtime exceptions, crashes, and sanitizer reports, without incorporating domain-specific numerical or semantic oracles that could expose deeper functional inconsistencies. Future extensions could enhance oracle diversity and sophistication to enable a broader spectrum of bug discovery.

Recent works have highlighted increasing concerns about LLM security, privacy, and potential data leakage risks (Zhou et al. 2025; Wang et al. 2025; Jaffal et al. 2025; Choi et al. 2025; Lin and Mohaisen 2025). It is worth noting that, to the best of our knowledge, there

Table 10 Prompt template for the API constraint analyzer

Role	Prompt content
System	You are an expert in deep learning libraries. Your task is to provide a comprehensive analysis report of API usage constraints through analyzing the API documentation and example code
User	Conduct constraint analysis for {api_name} from {dl_library} {version} with the following information: 1. API signature: {api_signature}; 2. API documentation: {api_doc}; 3. API usage code: {api_code}. Your report must include: (1) Parameter Analysis, (2) Inter-Parameter Constraints, (3) Valid/Invalid Combinations, (4) Output Specifications, (5) Contextual Dependencies, and (6) Package Requirements

Table 11 Prompt template for fuzz target function generation

Role	Prompt content
System	You are an expert in fuzz testing and DL library APIs. Your task is to design a Python 3 function <code>FuzzTarget(**kwargs)</code> that invokes a specific DL API and its dependencies. The function will be used as the fuzz target to detect vulnerabilities
User	Requirements of the target function: • Function must be named <code>FuzzTarget(**kwargs)</code> • Must invoke the target API: <code>{api_signature}</code> • Must include necessary pre/post conditions • Must ensure proper resource cleanup and termination • May include additional API calls to detect vulnerabilities in API interactions • All inputs and parameters MUST be derived from <code>kwargs</code> (no hardcoded values) Generate the target function with the following step-by-step instructions: 1. Design API call sequences 2. Identify any pre/post conditions 3. Identify any resource cleanup operations 4. Determine how to extract parameters from <code>kwargs</code> 5. Generate a complete fuzz target function in Python 3 following the suggested fuzz target structure

Table 12 Prompt template for fuzz data constructor generation

Role	Prompt content
System	You are an expert in fuzz testing and DL library APIs. Given target function <code>FuzzTarget(**kwargs)</code> , your task is to design a data constructor function <code>data_constructor(input)</code> in Python 3 to generate all the argument data required by the target function
User	Requirements of the data constructor function: • Function must be named <code>data_constructor(input)</code>. • Must accept a single argument <code>input</code>. • Primarily utilize the <code>FuzzedDataProvider</code> class provided by Atheris to generate random structured data from <code>input</code>. • Must include data validation code to ensure that all generated data complies with the constraints of the fuzz target function. • Must return a dictionary containing all the parameters and their corresponding values required for the target function. Generate the data constructor function with the following step-by-step instructions: 1. Identify all parameters required by the target function and their data structure/type. 2. Determine the constraints for each parameter. 3. Design data generation code. 4. Design the constraint validation code. 5. Map the generated parameters to the <code>kwargs</code> dictionary. 6. Complete the data constructor function in Python 3 following the suggested structure

Table 13 Prompt template for complete fuzz driver construction

Role	Prompt content
System	You are an expert in writing Atheris fuzz drivers. Your task is to design a complete fuzz driver in Python 3. You must complete the fuzz driver code by filling in the missing parts
User	Requirements of the fuzz driver: • Must include all necessary module imports. • Must instrument imported modules related to the tested DL libraries within the <code>atheris.instrument_imports()</code> block. • Must include the data constructor function and the target function. • Must implement an entry point function <code>entry_point(input)</code> to invoke the data construction function and the target function. • Must instrument functions with the <code>@atheris.instrument_func</code> decorator. • Must implement a <code>main()</code> function that sets up fuzzing environment. • Must include the <code>if __name__ == "__main__": main()</code> call. Generate the complete fuzz driver with the following step-by-step instructions: 1. Identify missing modules that need to be imported. 2. Identify modules that need to be instrumented. 3. Identify the code required for the entry point function. 4. Identify the fuzzing setup code. 5. Generate the complete fuzz driver code

Table 14 Prompt template for Ruff result analysis

Role	Prompt content
System	You are an expert in the static analysis tool Ruff. Your task is to analyze Ruff's automatic fixes and identify any issues that were detected but not successfully fixed
User	Identify any issues that were detected but not successfully fixed by Ruff. For each unfixed issue, provide: (1) Description of the problem, (2) Error message, (3) Error code snippet, (4) Location (code line number), (5) Root cause analysis, (6) Suggested fix (Preserve basic structure)

Table 15 Prompt template for fuzz driver execution result analysis

Role	Prompt content
System	You are a skilled software engineer and security analyst specializing in fuzz testing DL programs using the Atheris framework. Your task is to analyze the execution results of a fuzz driver program and identify potential issues that need to be addressed
User	Analyze the execution result and identify any issues in the fuzz driver. For each identified issue, provide: (1) Issues category, (2) Type/Description of the problem, (3) Relevant error/warning message, (4) Code location (fuzz driver code line number), (5) Root cause analysis, (6) Suggested fix (Preserve basic structure)

Table 16 Prompt template for fuzz driver coverage analysis

Role	Prompt content
System	You are a software testing expert specializing in analyzing code coverage for fuzz testing programs. Your task is to analyze the coverage report generated during a quick fuzz test and determine if the target API was executed. Identify design flaws in the fuzz driver that prevent coverage growth or fail to reach the target API
User	Check the fuzzing results and coverage data to determine whether the program shows any coverage growth and whether the target API {api_name} is covered. If not: 1. Identify design flaws in the fuzz driver that lead to the lack of coverage growth or API execution. 2. Provide repair suggestions

Table 17 Prompt template for fuzz driver decision making

Role	Prompt content
System	You are an expert in fuzz testing and program analysis specializing in DL library APIs. Your task is to evaluate the current state of a generated fuzz driver and make decisions about its viability
User	Based on the issue diagnosis information, make one of the five decisions: 1. REPAIR 2. ACCEPT 3. ACCEPT AND PRESERVE VULNERABILITY FINDINGS 4. ENVIRONMENT OPTIMIZATION 5. REJECT

Table 18 Prompt template for fuzz driver repair and optimization

Role	Prompt content
System	You are a senior Python programmer with deep knowledge of DL libraries and the Atheris fuzzing framework. Your task is to analyze and optimize a given Python fuzz driver that tests a DL library API for potential vulnerabilities. You must carefully address known design flaws while strictly maintaining all specified structural requirements of the original fuzz driver
User	Analyze the identified issues and suggested fixes, and provide a revised version of the fuzz driver that addresses all the issues while preserving the original structure and functionality. The revised fuzz driver must: 1. Maintain the original structure of the fuzz driver. 2. Address all the issues identified in the fix list. 3. Ensure that the fuzz driver is a complete and valid Python 3 code that can be executed

is currently a lack of large-scale studies that apply Atheris-based fuzzing to DL library API testing. Therefore, the generation of such fuzz drivers is unlikely to be affected by training-data leakage in existing LLMs. Furthermore, to minimize any potential bias from training data leakage, the generation process relies solely on official API documentation and source-level constraint analysis, rather than textual vulnerability descriptions, ensuring that detections result from genuine execution rather than memorized knowledge.

Conclusion

The growing use of DL in safety-critical scenarios has raised serious concerns about the security and robustness of its underlying frameworks. In this paper, we propose FD-FACTORY, a fully automated framework that leverages LLMs to generate reusable fuzz drivers for DL testing. Our approach tackles the key challenge of manual fuzz driver construction by utilizing LLMs' capabilities in natural language understanding, logical reasoning, code generation, error diagnosis, and error correction to fully automate the process. Unlike existing methods that rely on one-time, hard-coded test programs or require frequent interaction with LLM servers during testing, our framework decouples LLM usage from the testing phase to reduce computational overhead and communication latency for practical fuzzing. Specifically, the proposed method is organized into eight distinct stages: *Preparation, Initial Fuzz Driver Generation, Early Stop Checks, Verification, Issue Diagnosis, Decision Making, Repair Loop, and Deployment*. This modular design improves the overall generation success rate, achieving a 34.66% to 54.66% increase compared to prior approaches. It also supports comprehensive coverage analysis by supporting both Python and native C/C++ code. It achieves a total coverage of 308,351 lines on PyTorch and 528,427 lines on TensorFlow, significantly outperforming previous SOTA fuzzing techniques. Through its automation, flexibility, and high effectiveness, this work takes a substantial step toward scalable and secure testing of DL libraries.

Appendix A: Prompt templates

The prompt templates of each agent employed in this paper are provided in Appendix A, which are simplified due to space limitations (Tables 10, 11, 12, 13, 14, 15, 16, 17 and 18).

Acknowledgements

We appreciate all the editors and reviewers. This work is supported in part by the National Key R&D Program of China under Grants No. 2023YFB2704903 and No. 2020YFB1807504.

Author contributions

Tianming Zheng: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing—Original Draft. Fanchao Meng: Conceptualization, Methodology, Software. Ping Yi: Funding acquisition, Project administration, Resources, Supervision. Yue Wu: Funding acquisition, Project administration, Resources, Supervision, Writingreview & editing.

Funding

This work is supported in part by the National Key R&D Program of China under Grants No. 2023YFB2704903 and No. 2020YFB1807504.

Data availability

The datasets generated and/or analyzed during the current study are available in the Github repository.

Declarations

Competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential Conflict of interest: Yue Wu reports financial support was provided by National Key R&D Program of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Received: 10 October 2025 Accepted: 26 November 2025

Published online: 04 January 2026

References

- Archana R, Jeevaraj PE (2024) Deep learning models for digital image processing: a review. *Artif Intell Rev* 57(1):11
- Choi S, Alkinoon A, Alghuried A, Alghamdi A, Mohaisen D (2025) Attributing chatgpt-transformed synthetic code. In: 2025 IEEE 45th international conference on distributed computing systems (ICDCS), pp 89–99. IEEE
- Christou N, Jin D, Atlidakis V, Ray B, Kemerlis VP (2023) Ivysyn: Automated vulnerability discovery in deep learning frameworks. In: Calandrinio, J.A., Troncoso, C. (eds) 32nd USENIX security symposium, USENIX security 2023, Anaheim, CA, USA, pp 2383–2400. USENIX Association. <https://www.usenix.org/conference/usenixsecurity23/presentation/christou>
- Deng Y, Xia CS, Peng H, Yang C, Zhang L (2023) Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In: Just R., Fraser G. (eds) Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, ISSTA 2023, Seattle, pp 423–435. ACM. <https://doi.org/10.1145/3597926.3598067>
- Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2024) Large language models are edge-case generators: crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the 46th IEEE/ACM international conference on software engineering, ICSE 2024, Lisbon, Portugal, pp 70–17013. ACM. <https://doi.org/10.1145/3597503.3623343>
- Deng Y, Yang C, Wei A, Zhang L (2022) Fuzzing deep-learning libraries via automated relational API inference. In: Roychoudhury A, Cadar C, Kim M (eds) Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering, ESEC/FSE 2022, pp 44–56. ACM. <https://doi.org/10.1145/3540250.3549085>
- Fakhoury S, Naik A, Sakkas G, Chakraborty S, Lahiri SK (2024) Llm-based test-driven interactive code generation: user study and empirical evaluation. *IEEE Trans Softw Eng*
- Filus K, Domanska J (2023) Software vulnerabilities in tensorflow-based deep learning applications. *Comput. Secur.* 124:102948. <https://doi.org/10.1016/J.COSE.2022.102948>
- Force JT (2017) Security and privacy controls for information systems and organizations. Technical report, National Institute of Standards and Technology
- Google: google/atheris. [Online; accessed 2025-06-04] (2025). <https://github.com/google/atheris>

- GoogleCloud: applications of artificial intelligence (AI) | Google Cloud. [Online; accessed 2025-03-22] (2025). <https://cloud.google.com/discover/ai-applications>
- Gu J, Luo X, Zhou Y, Wang X (2022) Muffin: Testing deep learning libraries via neural architecture fuzzing. In: 44th IEEE/ACM 44th international conference on software engineering, ICSE 2022, Pittsburgh, pp 1418–1430. ACM. <https://doi.org/10.1145/3510003.3510092>
- Guo Q, Xie X, Li Y, Zhang X, Liu Y, Li X, Shen C (2020) Auddee: Automated testing for deep learning frameworks. In: 35th IEEE/ACM international conference on automated software engineering, ASE 2020, Melbourne, Australia, 2020, pp. 486–498. IEEE. <https://doi.org/10.1145/3324884.3416571>
- Huang D, Zhang JM, Bu Q, Xie X, Chen J, Cui H (2024) Bias testing and mitigation in llm-based code generation. *ACM Trans Softw Eng Methodol*
- Huang D, Zhang JM, Luck M, Bu Q, Qing Y, Cui H (2023) Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*
- Islam MA, Ali ME, Parvez MR (2024) Mapcoder: multi-agent code generation for competitive problem solving. In: Ku, L., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, pp 4912–4944. Association for Computational Linguistics. <https://doi.org/10.18653/V1/2024.ACL-LONG.269>
- Jaffal NO, Alkhanafseh M, Mohaisen D (2025) Large language models in cybersecurity: a survey of applications, vulnerabilities, and defense techniques. *AI* 6(9):216
- Joshi A, Dabre R, Kanojia D, Li Z, Zhan H, Haffari G, Dippold D (2025) Natural language processing for dialects of a language: A survey. *ACM Comput Surv* 57(6):1–37
- Kheddar H, Hemis M, Himeur Y (2024) Automatic speech recognition using advanced deep learning approaches: A survey. *Information Fusion*, 102422
- LazyProgrammer: PyTorch vs. TensorFlow: Full Overview 2025 Guide. [Online; accessed 2025-03-23] (2025). <https://lazyprogrammer.me/pytorch-vs-tensorflow/>
- Lin F, Kim DJ et al (2024) When llm-based code generation meets the software development process. *arXiv e-prints*, 2403
- Lin J, Mohaisen D (2025) From large to mammoth: a comparative evaluation of large language models in vulnerability detection. In: Proceedings of the 2025 network and distributed system security symposium (NDSS)
- Li J, Tao C, Li J, Li G, Jin Z, Zhang H, Fang Z, Liu F (2023) Large language model-aware in-context learning for code generation. *ACM Trans Softw Eng Methodol*
- Liu J, Huang Y, Wang Z, Ma L, Fang C, Gu M, Zhang X, Chen Z (2024) Generation-based differential fuzzing for deep learning libraries. *ACM Trans Softw Eng Methodol* 33(2):50–15028. <https://doi.org/10.1145/3628159>
- Liu F, Liu Y, Shi L, Huang H, Wang R, Yang Z, Zhang L, Li Z, Ma Y (2024) Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971*
- Nunez A, Islam NT, Jha SK, Najafirad P (2024) Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing. *arXiv preprint arXiv:2409.10737*
- Pham HV, Lutellier T, Qi W, Tan L (2019) CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries. In: Atlee, J.M., Bultan T, Whittle J (eds) Proceedings of the 41st international conference on software engineering, ICSE 2019, Montreal, QC, Canada, pp 1027–1038. IEEE/ACM. <https://doi.org/10.1109/ICSE.2019.00107>
- PyTorch: PyTorch. [Online; accessed 2025-03-23] (2025). <https://pytorch.org/>
- Ruff: Ruff-An extremely fast Python linter and code formatter, written in Rust. [Online; accessed 2025-05-26] (2025). <https://docs.astral.sh/ruff/>
- Tableau: everyday examples and applications of artificial intelligence (AI) | Tableau. [Online; accessed 2025-03-22] (2025). <https://www.tableau.com/data-insights/ai/examples>
- TensorFlow: TensorFlow. [Online; accessed 2025-03-23] (2025). <https://www.tensorflow.org/>
- Vidhya A (2023) Which are Common Applications of Deep Learning in AI? [Online; accessed 2025-03-22] . <https://www.analyticsvidhya.com/blog/2023/06/common-applications-of-deep-learning-in-artificial-intelligence/>
- Wang J, Ni T, Lee W-B, Zhao Q (2025) A contemporary survey of large language model assisted program analysis. *Trans Artif Intell* 1(1):105–129
- Wang Z, Yan M, Chen J, Liu S, Zhang D (2020) Deep learning library testing via effective model generation. In: Devanbu P, Cohen MB, Zimmermann T (eds) ESEC/FSE '20: 28th ACM joint European software engineering conference and symposium on the foundations of software engineering, Virtual Event, pp 788–799. ACM. <https://doi.org/10.1145/3368089.3409761>
- Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. *Adv Neural Inf Process Syst* 35:24824–24837
- Wei A, Deng Y, Yang C, Zhang L (2022) Free lunch for testing: Fuzzing deep-learning libraries from open source. In: 44th IEEE/ACM 44th international conference on software engineering, ICSE 2022, Pittsburgh, pp 995–1007. ACM <https://doi.org/10.1145/3510003.3510041>
- Xie D, Li Y, Kim M, Pham HV, Tan L, Zhang X, Godfrey MW (2022) Docter: documentation-guided fuzzing for testing deep learning API functions. In: Ryu S, Smaragdakis Y (eds) ISSTA '22: 31st ACM SIGSOFT international symposium on software testing and analysis, virtual event, South Korea, pp 176–188. ACM. <https://doi.org/10.1145/3533767.3534220>
- Yang C, Deng Y, Yao J, Tu Y, Li H, Zhang L (2023) Fuzzing automatic differentiation in deep-learning libraries. In: 45th IEEE/ACM international conference on software engineering, ICSE 2023, Melbourne, Australia, pp 1174–1186. IEEE. <https://doi.org/10.1109/ICSE48619.2023.00105>
- Zhang X, Jiang W, Shen C, Li Q, Wang Q, Lin C, Guan X (2025) Deep learning library testing: definition, methods and challenges. *ACM Comput Surv* 57(7):1–37
- Zhang C, Zheng Y, Bai M, Li Y, Ma W, Xie X, Li Y, Sun L, Liu Y (2024) How effective are they? exploring large language model based fuzz driver generation. In: Christakis M, Pradel M (eds) Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis, ISSTA 2024, Vienna, Austria, pp 1223–1235. ACM. <https://doi.org/10.1145/3650212.3680355>
- Zhao Z, Shi Y, Wu S, Yang F, Song W, Liu N (2023) Interpretation of time-series deep models: a survey. *arXiv preprint arXiv:2305.14582*
- Zhong L, Wang Z (2024) Can llm replace stack overflow? A study on robustness and reliability of large language model code generation. In: Proceedings of the AAAI conference on artificial intelligence vol 38, pp 21841–21849
- Zhou Y, Ni T, Lee W-B, Zhao Q (2025) A survey on backdoor threats in large language models (llms): attacks, defenses, and evaluation methods. *Trans Artif Intell* 3–3
- Zhou H, Wan X, Sun R, Palangi H, Iqbal S, Vulić I, Korhonen A, Arık S Ö (2025) Multi-agent design: Optimizing agents with better prompts and topologies. *arXiv preprint arXiv:2502.02533*

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.